



Android Java

6절. 네트워크 기초

❖ 네트워크

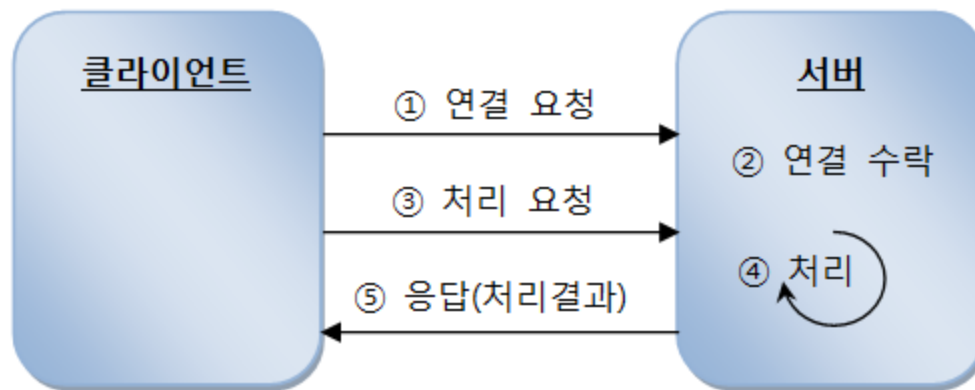
- 여러 대의 컴퓨터를 통신 회선으로 연결한 것
 - 홈 네트워크: 컴퓨터가 방마다 있고, 이들 컴퓨터를 유·무선 등의 통신 회선으로 연결
 - 지역 네트워크: 회사, 건물, 특정 영역에 존재하는 컴퓨터를 통신 회선으로 연결한 것
 - 인터넷: 지역 네트워크를 통신 회선으로 연결한 것



6절. 네트워크 기초

❖ 서버와 클라이언트

- 서버: 서비스를 제공하는 프로그램
 - 웹 서버, FTP서버, DBMS, 메신저 서버
 - 클라이언트의 연결을 수락하고, 요청 내용 처리한 후 응답 보내는 역할
- 클라이언트: 서비스를 받는 프로그램
 - 웹 브라우저, FTP 클라이언트, 메신저
 - 네트워크 데이터를 필요로 하는 모든 애플리케이션이 해당(모바일 앱 포함)



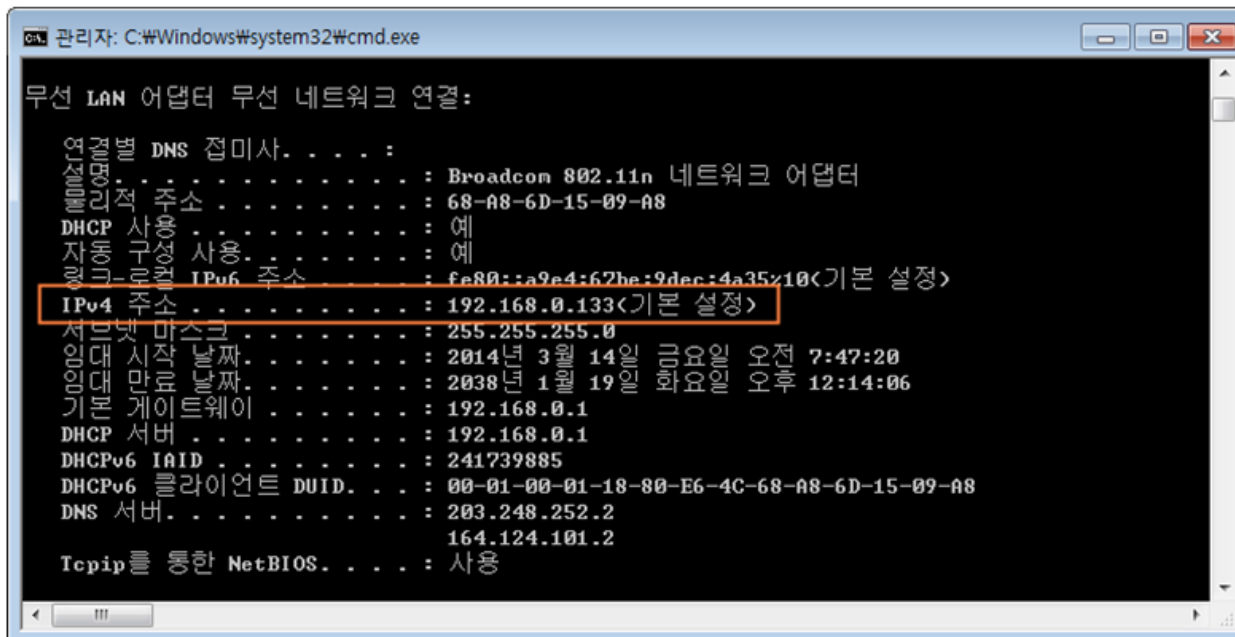
6절. 네트워크 기초

❖ IP 주소와 포트(port)

■ IP(Internet Protocol) 주소

- 네트워크상에서 컴퓨터를 식별하는 번호
- 네트워크 어댑터(랜 (Lan) 카드) 마다 할당
- IP 주소 확인 법 – 명령 프롬프트 (cmd.exe) 사용
- xxx.xxx.xxx.xxx 형식으로 표현 (xxx는 0~255 사이의 정수)

```
C:\W>ipconfig /all
```



```
관리자: C:\Windows\system32\cmd.exe

무선 LAN 어댑터 무선 네트워크 연결:

연결별 DNS 접미사. . . . . :
연결별 . . . . . :
물리적 주소 . . . . . : Broadcom 802.11n 네트워크 어댑터
DHCP 사용 . . . . . : 예
자동 구성 사용 . . . . . : 예
링크-로컬 IPv6 주소 . . . . . : fe80::a9e4:67be:9dec:4a35%10<기본 설정>
IPv4 주소 . . . . . : 192.168.0.133<기본 설정>
서브넷 마스크 . . . . . : 255.255.255.0
임대 시작 날짜 . . . . . : 2014년 3월 14일 금요일 오전 7:47:20
임대 만료 날짜 . . . . . : 2038년 1월 19일 화요일 오후 12:14:06
기본 게이트웨이 . . . . . : 192.168.0.1
DHCP 서버 . . . . . : 192.168.0.1
DHCPv6 IAD . . . . . : 241739885
DHCPv6 클라이언트 DUID. . . : 00-01-00-01-18-80-E6-4C-68-A8-6D-15-09-A8
DNS 서버 . . . . . : 203.248.252.2
                  164.124.101.2
Tcpip를 통한 NetBIOS. . . . : 사용
```

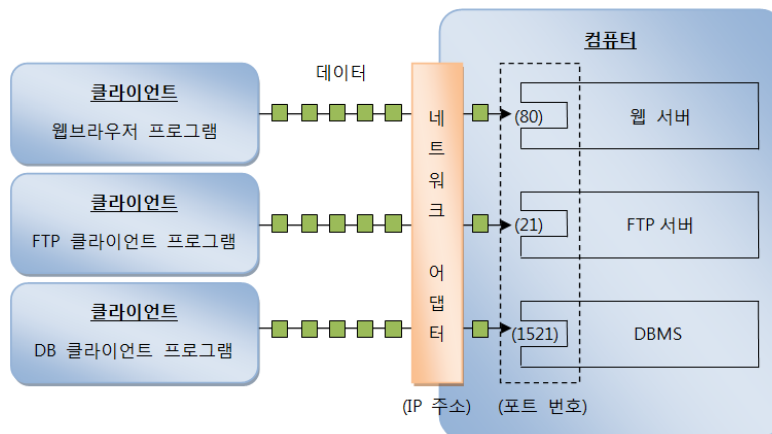


6절. 네트워크 기초

■ 포트(Port)

- 같은 컴퓨터 내에서 프로그램을 식별하는 번호
- 클라이언트는 서버 연결 요청 시 IP 주소와 Port 같이 제공
- 0~65535 범위의 값을 가짐
- 포트 범위는 세 가지로 구분

구분명	범위	설명
Well Know Port Numbers	0~1023	국제인터넷주소관리기구(ICANN)가 특정 애플리케이션용으로 미리 예약한 포트
Registered Port Numbers	1024~49151	회사에서 등록해서 사용할 수 있는 포트
Dynamic Or Private Port Numbers	49152~65535	운영체제가 부여하는 동적 포트 또는 개인적인 목적으로 사용할 수 있는 포트



6절. 네트워크 기초

❖ InetAddress로 IP 주소 얻기

■ java.net.InetAddress

- IP 주소 표현한 클래스
- 로컬 컴퓨터의 IP 주소
- 도메인 이름을 DNS에서 검색한 후 IP 주소를 가져오는 기능 제공



7절. TCP 네트워킹

❖ TCP(Transmission Control Protocol)

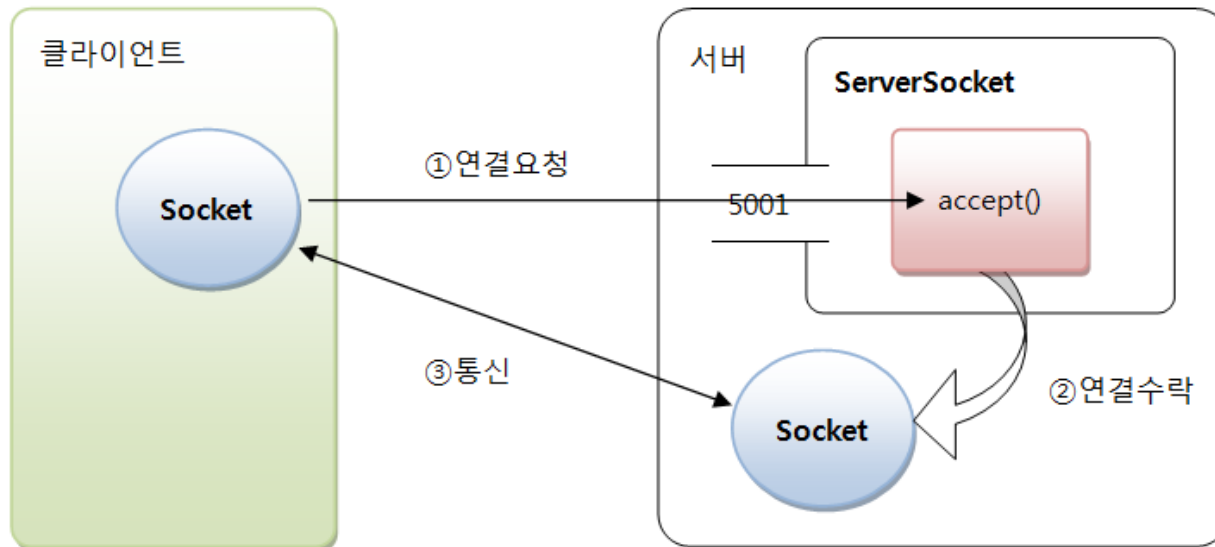
■ 특징

- 연결 지향적 프로토콜 -> 시간 소요
- 통신 선로 고정 -> 전송 속도 느려질 수 있음
- 데이터를 정확하고 안정적으로 전달

■ java.net API

- ServerSocket, Socket

■ ServerSocket과 Socket 용도



7절. TCP 네트워킹

❖ ServerSocket 생성과 연결 수락

■ ServerSocket 생성과 포트 바인딩

- 생성자에 바인딩 포트 대입하고 객체 생성

■ 연결 수락

- accept() 메소드는 클라이언트가 연결 요청 전까지 블로킹 → 대기
- 연결된 클라이언트 IP 주소 얻기

```
InetSocketAddress socketAddress = (InetSocketAddress) socket.getRemoteSocketAddress();
```

리터타입	메소드명(매개 변수)	설명
String	getHostName()	클라이언트 IP 리턴
int	getPort()	클라이언트 포트 번호 리턴
String	toString()	"IP:포트번호" 형태의 문자열 리턴

■ ServerSocket 포트 언바인딩

- 더 이상 클라이언트 연결 수락 필요 없는 경우



7절. TCP 네트워킹

❖ Socket 생성과 연결 요청

■ Socket 생성 및 연결 요청

- `java.net.Socket` 이용
- 서버의 IP 주소와 바인딩 포트 번호를 제공하면 생성과 동시에 사용가능

■ 연결 끊기

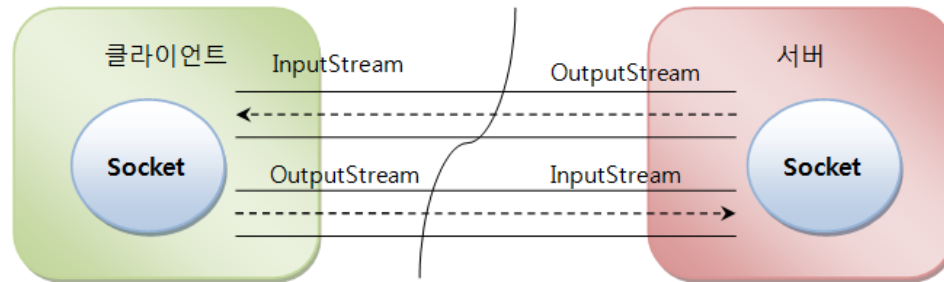
- Exception 처리 필요



7절. TCP 네트워킹

❖ Socket 데이터 통신

■ Socket 객체로 부터 입출력 스트림 얻기



- 입출력 스트림 구현 예제
 - 연결 성공 후 클라이언트가 서버에 “Hello Server”
 - 서버가 데이터 받음
 - 서버가 클라이언트에 “Hello Client” 보냄
 - 클라이언트가 데이터 받음
- read()의 블로킹 해제

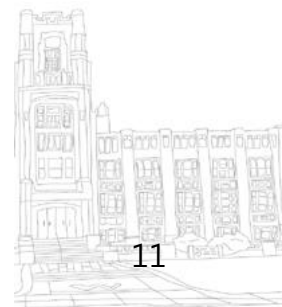
블로킹이 해제되는 경우	리턴값
상대방이 데이터를 보냄	읽은 바이트 수
상대방이 정상적으로 Socket 의 close()를 호출	-1
상대방이 비정상적으로 종료	IOException 발생

8절. UDP 네트워킹

❖ UDP(User Datagram Protocol)

■ 특징

- 비연결 지향적 프로토콜
 - 연결 절차 거치지 않고 발신자가 일방적으로 데이터 발신하는 방식
 - TCP 보다는 빠른 전송
- 통신 선로가 고정적이지 않음
 - 데이터 패킷들이 서로 다른 통신 선로 통해 전달될 수 있음
 - 먼저 보낸 패킷이 느린 선로 통해 전송될 경우, 나중에 보낸 패킷보다 늦게 도착 가능
- 데이터 손실 발생 가능성
 - 일부 패킷은 잘못된 선로로 전송되어 유실 가능
 - 데이터 전달 신뢰성 떨어짐



8절. UDP 네트워킹

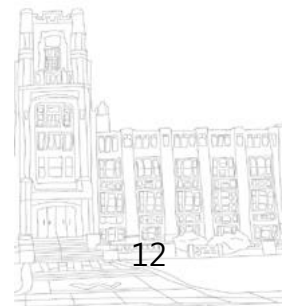
■ java.net API

- DatagramSocket, DatagramPacket



■ UDP 네트워킹 구현 예제

- 발신자 구현 코드 – 소켓 통해 데이터 패킷 전송
- 수신자 구현 코드 – 바인딩한 특정 포트로 데이터 받아 저장
- DatagramSocket 닫기





CHECK TIME

- 1 `http://docs.oracle.com/javase/7/docs/api/`를 나타내는 URL 객체를 생성하라.
- 2 1번에서 생성한 URL 객체에서 `URLConnection` 객체를 획득하는 코드를 작성하라.
- 3 2번에서 생성한 `URLConnection` 객체에서 입력 스트림을 얻어 와서 해당 URL의 내용을 콘솔에 출력하는 코드를 작성하라.

16.3 소켓 프로그래밍

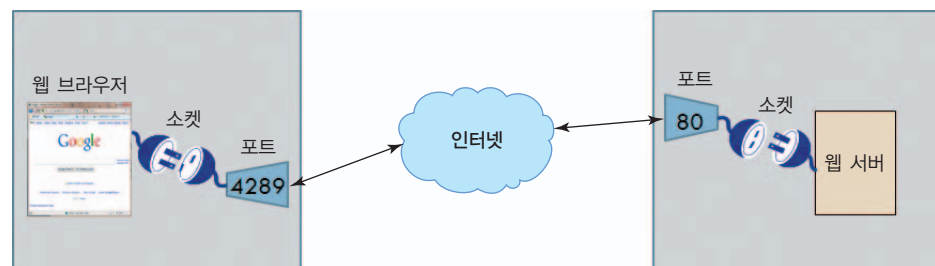


소켓

소켓
Socket

앞 절의 URL이나 `URLConnection` 객체를 이용한 웹 프로그래밍에서 서버와 통신할 때는 간단히 입출력 스트림을 생성하여 데이터를 주고받았다. 연결이 어떻게 설정되는지, 어떤 포트를 사용하는지, 서버와 클라이언트가 어떤 순서로 통신하는지 등 자세히 신경 쓸 필요가 없었다. 그러나 개발자가 서버와 클라이언트 사이의 통신을 보다 자세히 제어하거나 TCP 프로토콜상에서 동작하는 새로운 응용 프로토콜을 만들어 통신을 하길 원하는 경우에는 직접 **소켓**을 이용하여 프로그래밍을 해야 한다.

자바에서 소켓을 이용하여 데이터 통신을 하려면 소켓(Socket) 클래스를 이용한다. 소켓은 네트워크상에서 통신하는 두 응용프로그램 간의 양방향 통신 링크의 한쪽 끝을 의미한다. [그림 16-9]와 같이 소켓은 특정 포트 번호에 연결되어 있어 데이터를 보내거나 받을 때 해당 응용프로그램을 식별할 수 있다.



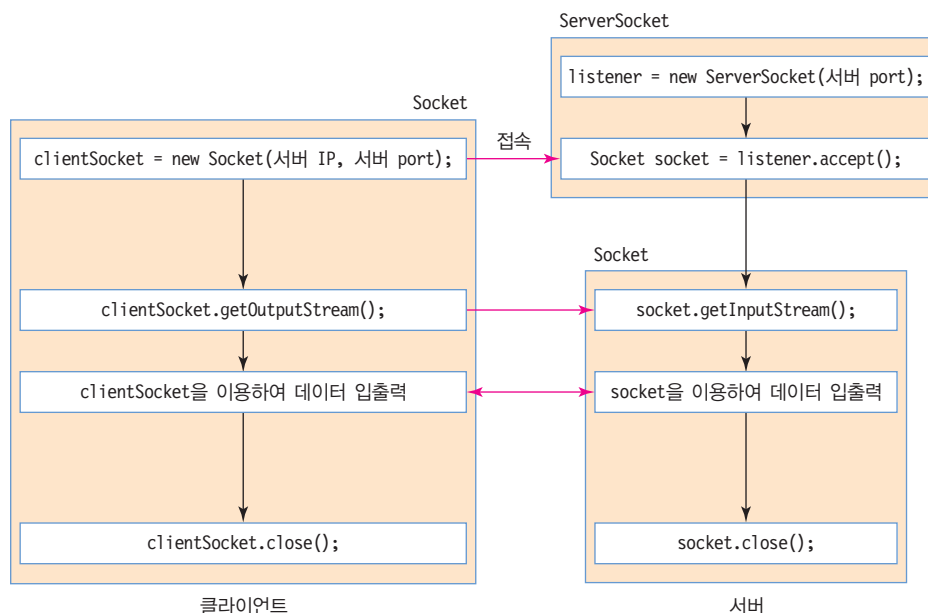
[그림 16-9] 소켓 이용

보통 정보를 제공하는 쪽을 **서버**라고 부르며, 이 정보를 이용하는 쪽을 **클라이언트**라고 부른다. 통신은 클라이언트에서 먼저 서버에 접속하여 서비스를 요청하는 방식으로 이루어지고, 서버는 데이터나 서비스를 제공하는 방식으로 이루어진다. 자바 소켓에는 서버 소켓과 클라이언트 소켓이 있다. **클라이언트 소켓**은 클라이언트를 위한 것이며, **서버 소켓**은 서버 응용프로그램을 위한 것이다.

서버
클라이언트
클라이언트 소켓
서버 소켓

소켓을 이용한 서버 클라이언트 통신 프로그램의 구조

이 절에서는 서버 클라이언트의 통신 구조를 전체적으로 묘사하고자 하며, 구체적인 내용은 다음 절에서 설명한다. 소켓을 이용하여 통신하는 서버 클라이언트의 전형적인 구조는 [그림 16-10]과 같다. 클라이언트 응용프로그램은 `Socket` 클래스를 이용하여 소켓 객체를 생성하고 서버와 통신하며, 서버 응용프로그램은 `ServerSocket` 클래스를 이용하여 소켓 객체를 생성하고 클라이언트와 통신한다. 서버는 클라이언트의 접속을 받기 위해 서버 포트에 대한 요청을 기다린다. 이때 클라이언트에서 서버 IP의 주소와 서버가 기다리는 포트 번호를 가지고 서버에 접속한다. 서버는 `accept()` 메소드에서 클라이언트와 통신하는 전용 소켓을 하나 따로 생성하여 접속된 클라이언트와 통신한다. 서버는 서버 소켓을 클라이언트와 통신하는 목적으로 사용하지 않고 클라이언트로부터의 접속을 받는 목적으로만 사용한다. 서버는 클라이언트가 접속할 때마다 `accept()` 메소드에서 따로 전용 소켓을 생성하여 클라이언트와 통신하도록 한다. 통신이 끝나면 소켓을 닫는다.



[그림 16-10] 소켓을 이용한 서버 클라이언트 통신 프로그램의 전형적인 구조



Socket

Socket 클래스, 클라이언트 소켓

Socket은 `java.net` 패키지에 포함되어 있는 클래스로서 클라이언트 소켓을 구현한다. 즉, 서버와 통신하기 위해 클라이언트 응용프로그램에서 사용하는 소켓이다. **Socket**의 생성자와 메소드는 각각 [표 16-3](#), [표 16-4](#)와 같다.

Socket 클래스의 생성자는 연결할 서버의 IP 주소(또는 도메인 주소)와 포트 번호를 인자로 받아서 새로운 **Socket** 객체를 생성한다. 클라이언트 자신의 주소와 포트 번호가 아님에 주의하자. 이제 **Socket** 클래스를 이용하여 클라이언트 응용프로그램을 작성하는 방법을 자세히 살펴보자.

● 클라이언트 소켓 생성 및 서버 접속

상대편 서버의 IP 주소가 128.12.1.1이고 상대편 포트 번호가 5550이면 다음과 같이 클라이언트 소켓 객체를 생성한다.

```
Socket clientSocket = new Socket("128.12.1.1", 5550);
```

Socket 객체가 생성되면 곧바로 128.12.1.1의 주소로 자동 접속이 이루어진다.

● 네트워크 입출력 스트림 생성

소켓이 만들어지고 서버와 연결이 된 후에는 **Socket** 클래스의 `getInputStream()`과 `getOutputStream()` 메소드를 이용하여 서버와 설정된 네트워크 연결을 다음과 같이 데이터 스트림으로 만든다.

```
BufferedReader in = new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));
BufferedWriter out = new BufferedWriter(new
    OutputStreamWriter(clientSocket.getOutputStream()));
```

`in`, `out` 객체를 이용하여 일반 스트림을 입출력하는 방식과 동일하게 네트워크 데이터터를 입출력한다.

● 서버로 데이터 전송

이제 앞서 만들어진 스트림 `out`을 통해 데이터를 전송한다. 다음은 “hello” 문자열을 서버로 전송하는 코드이다.

```
out.write("hello"+"\\n");
out.flush();
```

`out.flush()`를 호출하면 스트림 속의 데이터를 남기지 않고 즉각 모두 전송한다.

● 서버로부터 데이터 수신

앞서 만들어진 스트림 `in`을 통해 서버로부터 데이터를 수신할 수 있다. 다음은 각각 서버로부터 한 개의 문자를 입력받는 코드와 한 행의 문자열을 입력받는 코드이다.

```
int x = in.read(); // 서버로부터 한 개의 문자 수신
String line = in.readLine(); // 서버로부터 한 행의 문자열 수신
```

● 네트워크 접속 종료

데이터 송수신을 모두 수행하고 네트워크의 접속을 종료하고자 하면 다음과 같이 한다.

```
clientSocket.close();
```

생성자	설명
<code>Socket(InetAddress address, int port)</code>	소켓을 생성하여 지정된 IP 주소(address)와 포트 번호(port)에 연결한다.
<code>Socket(String host, int port)</code>	소켓을 생성하여 지정된 호스트(host)와 포트 번호(port)에 연결한다. 호스트 이름이 null인 경우는 루프백(loopback) 주소로 가정한다.

〈표 16-3〉

Socket 클래스의 주요 생성자

메소드	설명
<code>void close()</code>	소켓을 닫는다.
<code>void connect(SocketAddress endpoint)</code>	서버에 연결
<code>InetAddress getInetAddress()</code>	연결한 서버의 주소 리턴
<code>InputStream getInputStream()</code>	소켓에 대한 입력 스트림 리턴
<code>InetAddress getLocalAddress()</code>	소켓이 연결된 로컬 주소 리턴
<code>int getLocalPort()</code>	소켓이 연결된 로컬 포트 번호 리턴
<code>int getPort()</code>	소켓이 연결한 서버의 포트 번호 리턴
<code>OutputStream getOutputStream()</code>	소켓에 대한 출력 스트림 리턴
<code>boolean isBound()</code>	소켓이 로컬 주소에 연결되어 있으면 true 리턴
<code>boolean isConnected()</code>	소켓이 서버에 연결되어 있으면 true 리턴
<code>boolean isClosed()</code>	소켓이 닫혀 있으면 true 리턴
<code>void setSoTimeout(int timeout)</code>	데이터 읽기 타임아웃 시간 지정. 0이면 타임아웃 해제

〈표 16-4〉

Socket 클래스의 주요 메소드



ServerSocket 클래스, 서버 소켓

ServerSocket 클래스

ServerSocket 클래스는 서버 소켓을 구현한다. ServerSocket 클래스는 java.net 패키지에 포함되어 있으며 ServerSocket 클래스의 생성자와 메소드는 각각 <표 16-5>, <표 16-6>과 같다. ServerSocket은 클라이언트 측으로부터 연결 요청을 기다리는 모니터링을 위해 사용되며 서버가 클라이언트의 연결 요청을 수락하면 Socket 객체를 새로 생성한다. 이 Socket 객체를 이용하여 클라이언트와 통신하게 되므로, ServerSocket은 일반적인 데이터의 송수신에는 사용되지 않는다.

● 서버 소켓 생성

ServerSocket 클래스의 생성자는 사용할 포트 번호를 인자로 받아서 새로운 ServerSocket 객체를 생성한다. 이 포트 번호는 현재 자신의 포트 번호이다. 이미 사용 중인 포트 번호를 지정하면 오류가 발생한다. 5550번 포트를 사용하는 서버 소켓을 생성하는 예를 들면 다음과 같다.

```
ServerSocket serverSocket = new ServerSocket(5550);
```

● 클라이언트로부터 접속 기다림

ServerSocket 클래스의 accept() 메소드를 이용하여 클라이언트로부터의 연결 요청을 기다린다. accept() 메소드는 연결 요청이 오면 다음과 같이 새로운 Socket 객체를 리턴한다.

```
Socket socket = serverSocket.accept();
```

서버에서 클라이언트와의 데이터 통신은 새로 만들어진 Socket 객체를 통해서 이루어진다. 그러므로 ServerSocket 클래스는 Socket 클래스와 달리 주어진 연결에 대해 입출력 스트림을 만들어주는 메소드가 없다.

● 네트워크 입출력 스트림 생성

accept() 메소드에서 얻은 Socket 객체의 getInputStream()과 getOutputStream() 메소드를 이용하여 클라이언트와 설정된 네트워크 연결을 다음과 같이 데이터 스트림으로 만든다.

```
BufferedReader in = new BufferedReader(new
    InputStreamReader(socket.getInputStream()));
BufferedWriter out = new BufferedWriter(new
    OutputStreamWriter(socket.getOutputStream()));
```

`in`, `out` 객체를 이용하여 일반 스트림을 입출력하는 방식과 동일하게 네트워크 데이터를 입출력한다.

● 클라이언트로부터 데이터 수신

앞서 만들어진 스트림 `in`을 이용하여 클라이언트로부터 데이터를 수신할 수 있다. 다음은 각각 클라이언트로부터 한 개의 문자를 입력받는 코드와 한 행의 문자열을 입력받는 코드이다.

```
int x = in.read(); // 클라이언트로부터 한 개의 문자 수신
String line = in.readLine(); // 클라이언트로부터 한 행의 문자열 수신
```

● 클라이언트로 데이터 전송

앞서 만들어진 스트림 `out`을 통해 클라이언트로 데이터를 전송할 수 있다. 다음은 “Hi!, Client” 문자열을 클라이언트로 전송하는 코드이다.

```
out.write("Hi!, Client"+"\\n");
out.flush();
```

`out.flush()`를 호출하면 스트림 속의 데이터를 남기지 않고 즉각 모두 클라이언트로 전송한다.

● 네트워크 접속 종료

데이터 송수신을 모두 수행하고 네트워크의 접속을 종료하고자 하면 다음과 같이 한다.

```
socket.close();
```

● 서버 응용프로그램 종료

더 이상 클라이언트의 접속을 받지 않고 서버 응용 프로그램을 종료하고자 하는 경우 다음과 같이 `ServerSocket`을 종료시킨다.

```
serverSocket.close();
```

생성자	설명
<code>ServerSocket(int port)</code>	지정된 포트 번호(port)와 결합된 소켓을 생성한다.

〈표 16-5〉

`ServerSocket` 클래스의
주요 생성자



〈표 16-6〉
ServerSocket 클래스의
주요 메소드

메소드	설명
Socket accept()	연결 요청을 기다리다 요청이 들어오면 수락하고 Socket 객체 리턴
void close()	서버 소켓을 닫는다.
InetAddress getInetAddress()	서버 소켓에 연결된 로컬 주소 리턴
int getLocalPort()	서버 소켓이 연결 요청을 모니터링하는 포트 번호 리턴
boolean isBound()	서버 소켓이 로컬 주소에 연결되어 있으면 true 리턴
boolean isClosed()	서버 소켓이 닫혀 있으면 true 리턴
void setSoTimeout(int timeout)	accept()에 대한 타임아웃 시간 지정. 0이면 타임아웃 해제



잠깐!

통신 프로그램을 작성하기 위해서는 운영체제가 지원하는 TCP/IP 라이브러리를 굳이 사용할 필요가 없다. 소켓의 등장으로 인해 통신 프로그램 개발자는 매우 편리해졌다. 소켓은 자바뿐만 아니라 거의 모든 언어에 라이브러리로 지원되는 공통적인 개념이므로 자바의 소켓을 알면 다른 언어의 소켓 라이브러리 역시 이해하기 쉽다. 독자들이 통신 프로그램을 작성한다고 하면 일단 무조건 소켓을 생각하고 여기서부터 시작하면 된다.

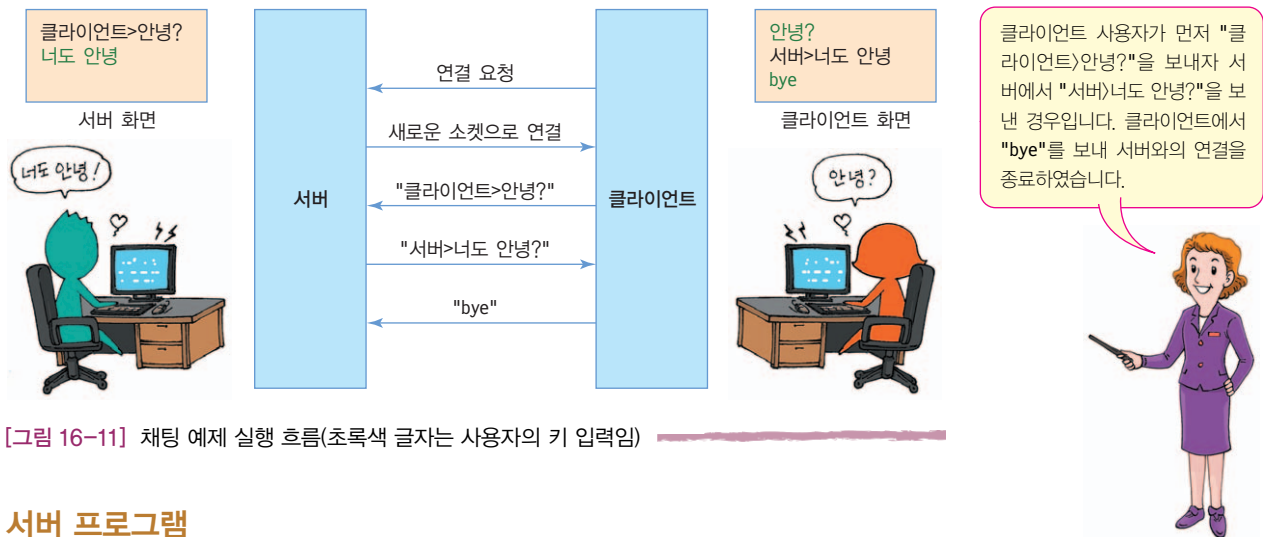
16.4 서버-클라이언트 채팅 예제



예제 개요

서버와 클라이언트가 번갈아 채팅 문자를 보내는 간단한 채팅 프로그램 예제를 통해 자바에서 소켓을 이용하여 어떻게 클라이언트와 서버 간에 데이터를 주고받는지 실습해보기로 한다. 본 예제에서 사용할 채팅 프로그램은 다음과 같다.

- 서버와 클라이언트가 1:1로 채팅
- 서버와 클라이언트 간의 메시지 구분을 위해 서버는 메시지 앞에 “서버>”를 접두어로 붙여 메시지를 전송하며, 클라이언트는 “클라이언트>”를 접두어로 붙여 메시지를 전송
- 서버와 클라이언트가 번갈아 가면서 메시지 전송 및 수신
- 클라이언트가 bye를 보내면 프로그램 종료





채팅과 같이 문자열을 송수신하는 경우는 문자 스트림을 사용하며, 파일을 전송하는 경우는 바이트 스트림을 사용한다. 본 예제에서는 효율적 입출력을 위하여 버퍼 스트림을 사용한다. 설정된 연결에서 스트림을 생성하는 코드는 다음과 같다.

```
BufferedReader in = new BufferedReader(new
    InputStreamReader(socket.getInputStream()));
BufferedWriter out = new BufferedWriter(new
    OutputStreamWriter(socket.getOutputStream()));
```

● 클라이언트로부터 데이터 수신

in 스트림 생성 이후부터는 일반 입력 스트림에서 데이터를 입력받는 방법과 동일하다. 다음 코드는 입력 스트림 in에서 한 행의 데이터를 수신하는 코드이다.

```
String inputMessage = in.readLine();
```

클라이언트에서 한 행의 문자열을 보내올 때까지 서버는 어떤 동작도 하지 않고 기다린다.

● 클라이언트로 데이터 송신

서버에서 클라이언트로 데이터를 보내는 방법은 일반 출력 스트림에서 데이터를 출력하는 방법과 동일하다. 아래의 코드는 콘솔에서 사용자로부터 문자열을 입력받아 클라이언트로 송신하는 코드이다.

```
BufferedReader stin = new BufferedReader(new InputStreamReader(System.in));
String outputMessage = stin.readLine(); // 콘솔에서 읽는다.
out.write("서버> " + outputMessage + "\n"); // 클라이언트로 전송한다.
out.flush();
```

flush() 메소드는 스트림 out에 남아 있는 모든 데이터를 클라이언트로 송신한다.

● 연결 종료

모든 데이터의 송수신이 끝나서 더 이상의 클라이언트와 서버의 연결이 필요하지 않은 경우에는 소켓을 닫는다. 소켓을 닫으면 소켓의 입출력 스트림도 같이 닫힌다. 서버 소켓을 닫아버리면 더 이상의 클라이언트 연결 요청을 받을 수 없다는 것에 유의한다.

다음의 코드는 소켓과 서버 소켓을 닫는 코드이다.

```
socket.close(); // 통신 소켓을 닫는다.  
listener.close(); // 서버 소켓을 닫는다.
```

소켓 또는 서버 소켓을 닫으면 여기에 관련된 모든 자원이 해제된다.

클라이언트 프로그램

이 절에서는 단계별로 클라이언트 프로그램 작성에 필요한 내용을 소스와 함께 설명한다.

● 연결 요청

클라이언트의 연결 요청이 있으면 연결할 서버의 호스트 주소와 서버의 포트 번호로 클라이언트 소켓을 생성한다. 다음은 클라이언트 소켓 생성 코드이다.

```
socket = new Socket("localhost", 9999);
```

본 예제에서는 동일한 컴퓨터 내에서 서버와 클라이언트가 동작하므로 호스트 이름으로 “localhost”로 지정하였다. “localhost”는 현재 프로그램이 실행되는 컴퓨터를 의미한다. 만약 서버가 다른 컴퓨터에서 동작한다면 서버의 IP 주소 또는 도메인 이름을 지정하면 된다.

● 서버와 통신을 위한 입출력 스트림 생성

서버와의 데이터 송수신을 위해 스트림을 생성한다. 클라이언트 측에서도 송수신되는 데이터의 종류에 따라 바이트 스트림 또는 문자 스트림을 생성하여 사용한다. 이 예제에서는 효율적 입출력을 위하여 버퍼 스트림을 사용한다. 클라이언트 소켓으로부터 입출력을 위한 스트림을 생성하는 코드는 다음과 같다.

```
BufferedReader in = new BufferedReader(new  
    InputStreamReader(socket.getInputStream()));  
BufferedWriter out = new BufferedWriter(new  
    OutputStreamWriter(socket.getOutputStream()));
```

● 서버에 데이터 송신

스트림 생성 이후는 일반 출력 스트림에서 데이터를 출력하는 방법과 동일하다. 다음 코드는 콘솔에서 입력받은 문자열을 서버로 송신하는 코드이다.



```
BufferedReader stin = new BufferedReader(new InputStreamReader(System.in));
String outputMessage = stin.readLine(); // 콘솔에서 읽는다.
out.write("클라이언트>" + outputMessage + " \n"); // 서버로 전송한다.
out.flush();
```

flush() 메소드는 스트림에 남아 있는 모든 데이터를 서버로 송신한다.

● 서버로부터 데이터 수신

스트림 생성 이후는 일반 입력 스트림에서 데이터를 입력받는 방법과 동일하다. 다음의 코드는 서버에서 한 행의 문자열을 보내올 때까지 어떤 동작도 하지 않고 기다리는 것을 나타낸다.

```
String inputMessage = in.readLine();
```

● 연결 종료

모든 데이터의 송수신이 끝나서 더 이상의 서버와 연결이 필요하지 않은 경우에는 소켓을 닫는다. 소켓을 닫으면 소켓의 입출력 스트림도 같이 닫힌다. 다음의 코드는 소켓을 닫는 코드이다.

```
socket.close();
```

소켓을 닫으면 여기에 관련된 모든 자원이 해제된다.

클라이언트 서버 채팅 소스 코드

예제 전체 소스 코드는 다음과 같다.

• 서버 프로그램 ServerEx.java

```
1 import java.io.*;
2 import java.net.*;
3
4 public class ServerEx {
5     public static void main(String[] args) {
6         BufferedReader in = null;
7         BufferedReader stin = null;
```

```
8      BufferedWriter out = null;
9      ServerSocket listener = null;
10     Socket socket = null;
11     try {
12         listener = new ServerSocket(9999); // 서버 소켓 생성
13         socket = listener.accept(); // 클라이언트로부터 연결 요청 대기
14         System.out.println("연결됨");
15         in = new BufferedReader(new InputStreamReader(socket.
16                                     getInputStream())); // 클라이언트로부터의 입력 스트림
17         stin = new BufferedReader(new InputStreamReader(System.in)); // 키보드
18                                    로부터의 입력 스트림
19         out = new BufferedWriter(new OutputStreamWriter(socket.
20                                     getOutputStream())); // 클라이언트로의 출력 스트림
21         String inputMessage;
22         while (true) {
23             inputMessage = in.readLine(); // 클라이언트에서 한 행의 문자열 읽음
24             if (inputMessage.equalsIgnoreCase("bye")) // 클라이언트가 "bye"를 보
25                                     내면 연결 종료
26                 break;
27             System.out.println(inputMessage); // 클라이언트가 보낸 메시지 화면에 출력
28             String outputMessage = stin.readLine(); // 키보드에서 한 행의 문자열 읽음
29             out.write("서버> " + outputMessage + "\n"); // 키보드에서 읽은 문자열 전송
30             out.flush();
31         }
32     } catch (IOException e) {
33         System.out.println(e.getMessage());
34     } finally {
35         try {
36             socket.close(); // 클라이언트와 통신용 소켓 닫기
37             listener.close(); // 서버 소켓 닫기
38         } catch (IOException e) {
39             System.out.println("클라이언트와 채팅 중 오류가 발생했습니다.");
40         }
41     }
42 }
```




• 클라이언트 프로그램 ClientEx.java

```

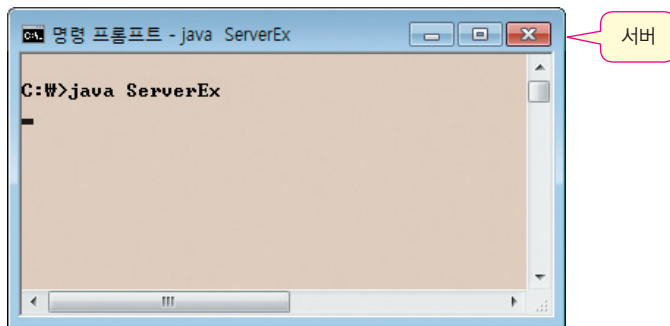
1  import java.io.*;
2  import java.net.*;
3
4  public class ClientEx {
5      public static void main(String[] args) {
6          BufferedReader in = null;
7          BufferedReader stin = null;
8          BufferedWriter out = null;
9          Socket socket = null;
10         try {
11             socket = new Socket("localhost", 9999); // 클라이언트 소켓 생성
12             in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
13                                                         // 서버로부터의 입력 스트림
14             stin = new BufferedReader(new InputStreamReader(System.in)); // 키보드
15                                                         //로부터의 입력 스트림
16             out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
17                                                         // 서버로의 출력 스트림
18
19             String outputMessage;
20             while (true) {
21                 outputMessage = stin.readLine(); // 키보드에서 한 행의 문자열 읽음
22                 if (outputMessage.equalsIgnoreCase("bye")) { // "bye"가 입력되면 메시
23                                                         //지 전송 후 연결 종료
24                     out.write(outputMessage);
25                     out.flush();
26                     break;
27                 }
28                 out.write("클라이언트>"+outputMessage+"\n"); // 키보드에서 읽은 문자열 전송
29                 out.flush();
30                 String inputMessage = in.readLine(); // 서버에서 한 행의 문자열 읽음
31                 System.out.println(inputMessage); // 서버가 보낸 메시지 화면에 출력
32             }
33         } catch (IOException e) {
34             System.out.println(e.getMessage());
35         } finally {
36             try {
37                 socket.close(); // 클라이언트 소켓 닫기
38             } catch (IOException e) {
39                 System.out.println("서버와 채팅 중 오류가 발생했습니다.");
40             }
41         }
42     }
43 }

```

서버-클라이언트 채팅 예제 실행

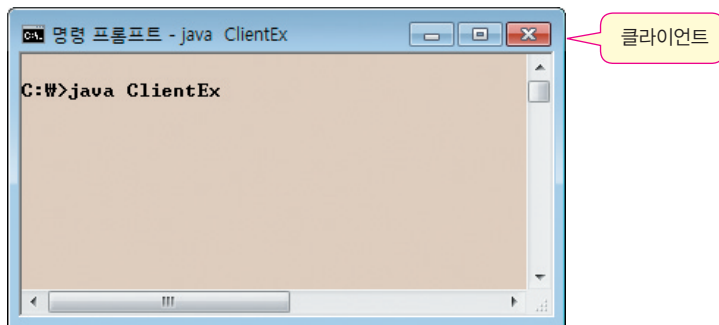
이제 만든 예제를 실행해보자. 예제를 실행하기 위해서는 서버와 클라이언트를 위해 각각 명령창을 하나씩 생성한다. 다음과 같은 순서로 진행한다.

1. 명령창을 하나 열고 서버 프로그램을 먼저 실행시켜 클라이언트의 연결 요청을 기다린다. [그림 16-12]와 같다.

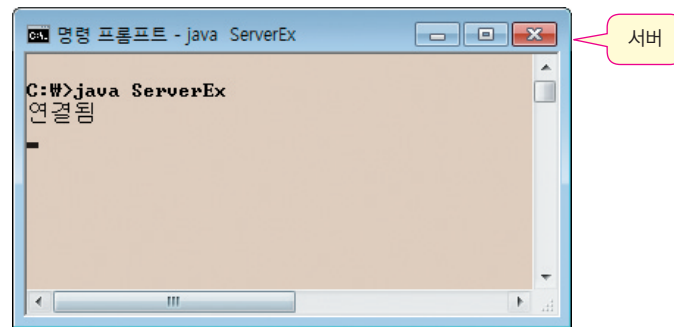


[그림 16-12] 서버가 먼저 실행되어 클라이언트로부터 연결을 기다리고 있는 서버 명령창

2. 서버 프로그램을 실행한 후에 [그림 16-13]과 같이 다시 다른 명령창을 열어 클라이언트 프로그램을 실행하고 서버와 연결을 설정한다. 연결이 설정되면 [그림 16-14]와 같이 서버의 명령창에는 “연결됨”이 출력된다.

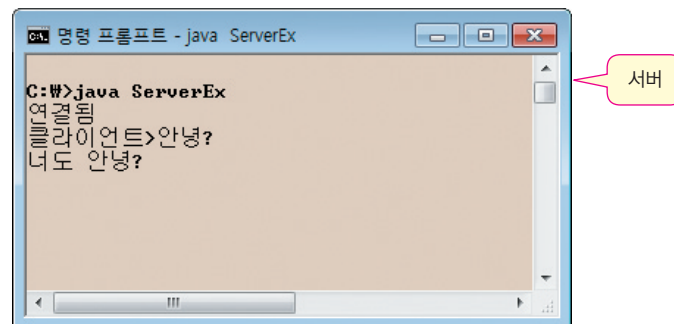


[그림 16-13] 클라이언트 프로그램을 실행한 클라이언트 명령창

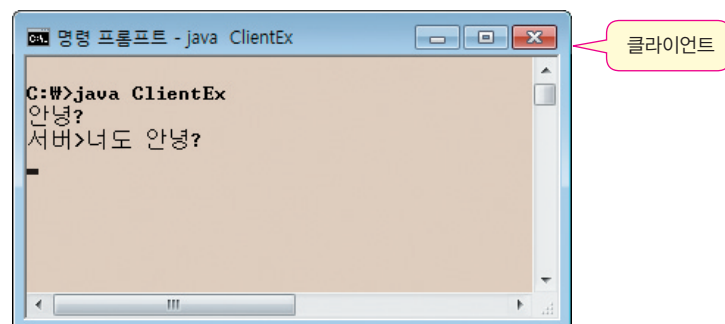


[그림 16-14] 클라이언트와 접속된 서버 명령창

3. 채팅은 클라이언트에서 먼저 시작하여 순서대로 메시지를 번갈아 주고받는다. [그림 16-15], [그림 16-16]은 각각 클라이언트와 서버의 실행 화면이다. 클라이언트 사용자가 “안녕?”을 콘솔에 입력하면 “클라이언트>안녕?”을 서버로 전송하고 서버에서는 받은 문자열을 그대로 출력한다. 그리고 [그림 16-15]와 같이 서버에서 사용자로부터 입력받은 “너도 안녕?” 문자열에 “서버>”를 덧붙여 클라이언트로 전송한다.



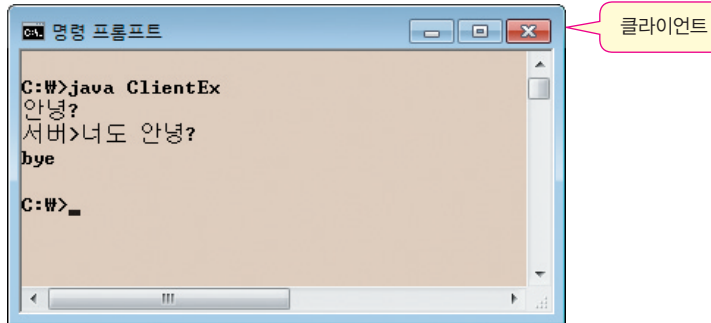
[그림 16-15] 클라이언트로부터 “클라이언트>안녕?”을 받아 출력하고 사용자로부터 메시지를 입력받아 전송하는 서버 명령창



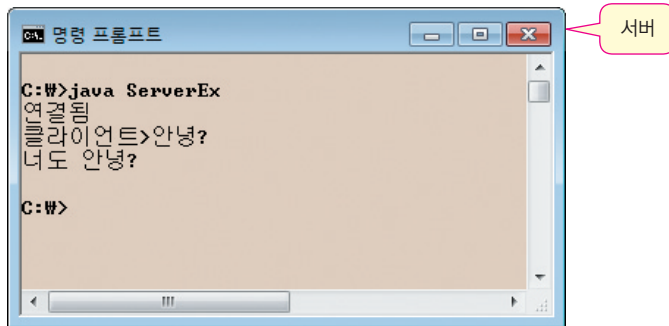
[그림 6-16] 서버로부터 “서버>너도 안녕?” 메시지를 전송받아 출력하는 클라이언트 명령창

4. 클라이언트에서 “bye”를 입력하여 채팅 종료

채팅을 종료하기 위해서는 클라이언트에서 “bye”를 서버에게 전송하면 된다. 사용자가 “bye”를 입력하면 이 문자열을 그대로 서버에게 보내 채팅을 종료한다. [그림 16-17]은 “bye” 문자열을 입력받아 서버로 보내는 과정이다. [그림 16-18]은 클라이언트로부터 “bye” 문자열을 받고 종료한 서버의 화면이다.



[그림 16-17] 클라이언트가 사용자로부터 “bye”를 입력받고 서버로 전송. 클라이언트 종료



[그림 16-18] 클라이언트로부터 “bye”를 전송받아 서버 프로그램 종료

- 1 서버 소켓에 스트림을 생성하는 메소드가 없는 이유를 설명하라.
- 2 연결이 설정된 클라이언트 소켓에서 바이트 입출력 스트림을 생성하는 코드를 작성하라.

